Exploring Collaboration Patterns in GitHub using Discrete Time Markov Chain

Suhee Jo
dept. of software safety and cyber security
University of Kyonggi
Suwon, Republic of Korea
sujo@kyonggi.ac.kr

Ryeonggu Kwon
dept. computer science
University of Kyonggi
Suwon, Republic of Korea
rkkwon@kyonggi.ac.kr

Gihwon Kwon
dept. computer science
University of Kyonggi
Suwon, Republic of Korea
khkwon@kyonggi.ac.kr

2023 30th Asia-Pacific Software Engineering Conference (APSEC) | 979-8-3503-4417-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/APSEC60848.2023.00090

***Abstract*—Collaboration among team members plays a crucial role in the success of software projects. GitHub has emerged as a prominent platform for collaborative software development. This study focuses on analyzing GitHub repositories to gain insights into project status and participant characteristics based on development activities such as commits and pull requests. Utilizing a discrete time Markov chain, this research models GitHub repositories and employs probabilistic temporal logic and model checking to shed light on the activity levels of both projects and participants. We expect that understanding these observed trends and patterns can assist project managers in strengthening teamwork and fostering productive collaboration.**

***Index Terms*—software project management, discrete time markov chain, model checking, github**

I. Introduction

The failure of software projects can result in significant time and cost losses [1]. To minimize the possibility of such failures and successfully complete projects, effective management is essential, including understanding project characteristics and encouraging teamwork among team members. Various software management tools have been introduced, primarily focusing on ensuring project completeness and traceability using frameworks like Kanban boards or Gantt charts. However, these tools face challenges in capturing and analyzing the dynamics of teamwork and collaboration within the project. In this research, we aim to address this by analyzing development activities based on actions.

In this study, we analyze projects based on GitHub. In the context of GitHub, development activities refer to a series of actions that individuals take to contribute from their local repositories to a remote repository. These actions include **creating** branches, **committing** changes to record them, creating **pull requests** to **merge** completed features into other branches, and **accepting** or **rejecting** these requests [2].

We create a DTMC (Discrete-Time Markov Chain) [3] model based on a combination of participants, branch positions and development actions as states. We perform model checking [4] by applying property specifications based on probabilistic temporal logic to this model to verify the desired insights. By quantifying the probabilities and patterns of development activities, we can identify areas in project and team management that require improvement. We expect that this will provide valuable insights for software project management.

II. Methodology

Data Collection. This study aims to analyze the contributions and activities of team members for project management. To achieve this, we selected projects that are relatively small in scale and have reached a certain level of progress. For data collection, we used PyGitHub [5] to collect samples, and the selection criteria for the samples were as follows: 1) Repositories created in 2022, 2) 3 to 10 contributors, 3) At least 100 commit logs. Following these criteria, we selected 30 repositories and extracted a total of 173 participants and over 11,800 activity logs.

State Structures. To model using DTMC, it is essential to define states. In this research, states are composed of a 3-tuple of participant, branch and action. *Participant* refers to individuals involved in the project and are assigned sequentially from $p1$ to pn for anonymity. *Branch* indicates the location of the branch where actions were performed, with "master" representing the default branch and other branches denoted as "branch". *Action* encompasses six types of development activities: "create", "commit", "pullRequest", "rejected", "accepted" and "merge". Each state is composed of a combination of $participant_branch_action$, which can be interpreted as "$participant$ do $action$ at $branch$". Additionally, states labeled as "Start" were used to mark the beginning of each month.

DTMC Modeling. The next step involves converting the extracted logs into the defined state structure and arranging them chronologically. Based on the sorted state logs, we calculate the probabilities of transitioning from one state to another, generating a transition matrix. At this point, the basic model is completed. We add labels and rewards for use. Labels encompass all combinations of tuples. For example, the label "commit" denotes all states that include "commit". The basic structure of rewards involves prefixing the label with "r_". The reward structure used in this study is as follows: r_Steps (1 point for each transition), r_participant_commit (1 point each time the participant commits).

We use the PRISM [6] for model checking. We model each of the 30 repositories using the proposed method, resulting in 30 DTMC models. The subsequent sections will present the results of the model checking.

2640-0715/23/$31.00 ©2023 IEEE
DOI 10.1109/APSEC60848.2023.00090

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:59:25 UTC from IEEE Xplore. Restrictions apply.

III. Experimental Results

Q1. What activities are most likely in the long run?

$$S = ?[\,actionLabel\,]$$

This query utilizes the S operator to calculate the long-term probabilities of each action taking place within the model. Fig. 1 displays the model checking results for each model. The average probabilities for each action are as follows: 8.04% for "create", 62.73% for "commit", 9.52% for "pullRequest", 1.00% for "rejected", 8.21% for "accepted", and 8.21% for "merge". These probabilities represent the expected proportions of each action occurring in the long run. These results provide insights into the activity trends for each project.

Q2. What is the average number of activities performed in a month?

$$R\{\text{“}r_Steps\text{”}\} = ?[X\ F\ \text{“}Start\text{”}]$$

This query is used to calculate the average activity time steps within a month using the transition reward "r_Steps". This query employs the X and F properties to calculate the average cumulative reward until reaching the "Start" label, which represents the beginning of each month. That is, this query provides results for the average number of activities occurring within a month. Fig. 2-(a) presents the results obtained from the model checking process, illustrating the average activity time steps within a month for the entire model. The average value of 85.31 suggests that, on average, there were approximately 85 activities performed within a month. This metric provides project managers with an understanding of the level of activity and engagement within the project during each month.

Q3. How long does it take from one participant to another?

$$filter(avg,\ R\{\text{“}r_Steps\text{”}\} = ?[F\,!\text{“}participant\text{”}],\ \text{“}participant\text{”})$$

This query calculates the average time steps it takes from one participant to another participant appearing. This metric provides insights into the dynamics of participant involvement and collaboration within the project. A smaller average time step indicates consistent and frequent participation from various team members, while a larger time step suggests longer periods of working alone. To obtain the results, the query was applied to each participant in the model, and the average time steps were calculated. Fig. 2-(b) shows the results obtained from the model checking.

IV. Conclusions

This paper presents a novel approach to software management that emphasizes the analysis of development activity flows using Markov models and property specification for model checking. While traditional software management tools tend to focus on optimizing code quality and build time, this approach provides a more in-depth understanding of the development process by examining the interactions between team members.

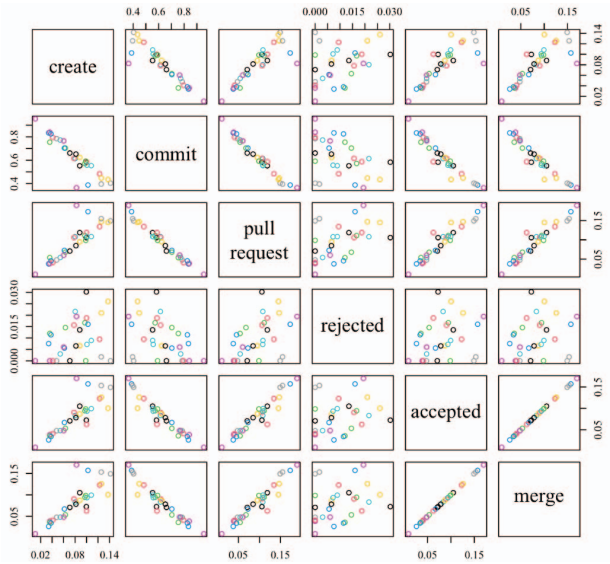

Fig. 1. Results of Q1

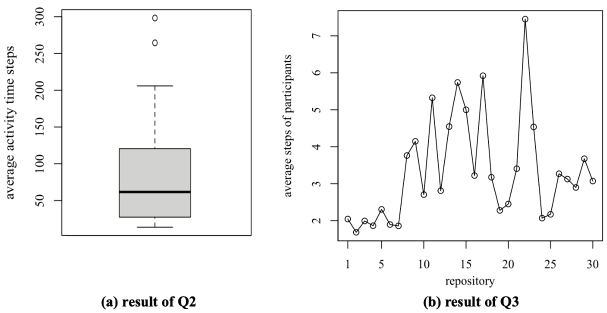

Fig. 2. Results of Q2 and Q3

Acknowledgment

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2021-0-00122, Safety Analysis and Verification Tool Technology Development for High Safety Software Development)

References

[1] C. Jones, "Software project management practices: Failure versus success," CrossTalk: The Journal of Defense Software Engineering, vol. 17, no. 10, pp. 5–9, 2004
[2] D. Spinellis, "Git," IEEE software, vol. 29, no. 3, pp. 100–101, 2012.
[3] N. Privault, "Discrete-time Markov chains," Understanding Markov Chains: Examples and Applications, pp. 89–113, 2018.
[4] M. Kwiatkowska, G. Norman and D. Parker, "Stochastic model checking", in Formal Methods for Performance Evaluation, Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 220–270.
[5] pyGithub, https://pygithub.readthedocs.io/en/latest/ (accessed Oct. 15, 2023)
[6] M. Kwiatkowska, G. Norman and D. Parker, "PRISM 4.0: Verification of Probabilistic Real-Time Systems," International Conference on Computer Aided Verification(CAV), 2011, pp. 585–591.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:59:25 UTC from IEEE Xplore. Restrictions apply.